

# **Dementia Stage Classification from Brain MRI Using Classical Machine Learning and a Convolutional Neural Network**

**Kyle Lunsford**

Machine Learning — Final Project

ITCS 3156, Spring 2026

Instructor: Hongfei Xue

# 1. Introduction

---

**Personal motivation.** I chose this project for a personal reason: my dad lives with dementia, and watching what the disease does to a family up close has made me want to contribute to the fight against it however I can. Machine learning is one of the most promising tools currently being applied to early detection and staging of neurodegenerative disease, and being able to spend a semester project on something so directly tied to my family — even at a small student scale — felt like the right way to start. Long-term, I hope to keep working on problems in this space.

**Problem statement.** Dementia — most commonly caused by Alzheimer’s disease — is a progressive neurodegenerative condition marked by measurable structural changes in the brain such as cortical atrophy, widening of cortical sulci, and enlargement of the lateral ventricles. Magnetic Resonance Imaging (MRI) captures these changes non-invasively and is a standard clinical tool for dementia assessment. In this project I build a machine-learning pipeline that takes a single axial brain-MRI slice as input and predicts one of four ordered severity classes: `NonDemented`, `VeryMildDemented`, `MildDemented`, or `ModerateDemented`.

**Motivation and challenges.** Manual MRI review is expensive and bottlenecked by radiologist availability, and subtle stage-to-stage differences are hard for even trained humans to detect consistently (Frisoni et al.). Automated staging of MRI slices could support screening, triage, and research cohort selection. The problem is not easy: adjacent severity stages look very similar, the underlying classes are clinically ordinal (rather than categorical), and the four classes in the dataset are imbalanced. A further challenge is that a single 2D slice contains only a small fraction of the information in a full 3D MRI volume.

**Concise summary of approach.** I treated the problem two ways and compared them head-to-head on the same train / validation / test split. The first approach uses material directly from our course: I extract 15 hand-crafted image features (brightness, contrast, edge density, an 8-bin intensity histogram, and four GLCM texture statistics) and train three classical classifiers on them — Random Forest, Histogram Gradient Boosting, and Logistic Regression. While I was working on the project I kept reading that the state of the art on MRI tasks is actually convolutional neural networks (CNNs). CNNs are not part of our course, but I was curious enough that I decided to teach myself the basics and try one as a second approach, training a small custom CNN directly on raw pixels. Comparing the two tests an important hypothesis: classical features discard spatial structure, so a CNN that sees the full spatial image *should* do substantially better if spatial structure is the dominant signal. The full pipeline is implemented in a single Jupyter notebook in Python 3.13 using NumPy, SciPy, scikit-learn, scikit-image, OpenCV, PyTorch, Matplotlib, and Seaborn (Harris et al.; Pedregosa et al.; van der Walt et al.; Bradski; Paszke et al.; Hunter; Waskom).

## 2. Data

### 2.1 Dataset

The dataset used is Aryan Singhal’s publicly available *Alzheimer’s Multiclass Dataset (Equal and Augmented)* on Kaggle, a collection of preprocessed axial brain-MRI slices organized into four class folders (MildDemented, ModerateDemented, NonDemented, VeryMildDemented) (Singhal). In total the dataset contains **approximately 44,000 JPEG images** at  $200 \times 190$  pixels, which easily exceeds the project’s 10,000-sample requirement. The per-class counts are roughly NonDemented  $\approx 12,800$ , VeryMildDemented  $\approx 11,200$ , MildDemented  $\approx 10,000$ , and ModerateDemented  $\approx 10,000$ . The majority-class baseline accuracy is therefore about 29 % — the floor any real model must clear.

### 2.2 Visual exploration

Figure 1 shows the exact class distribution as computed from the data. The imbalance is modest but present, motivating the use of stratified splits and class-balanced weighting downstream.

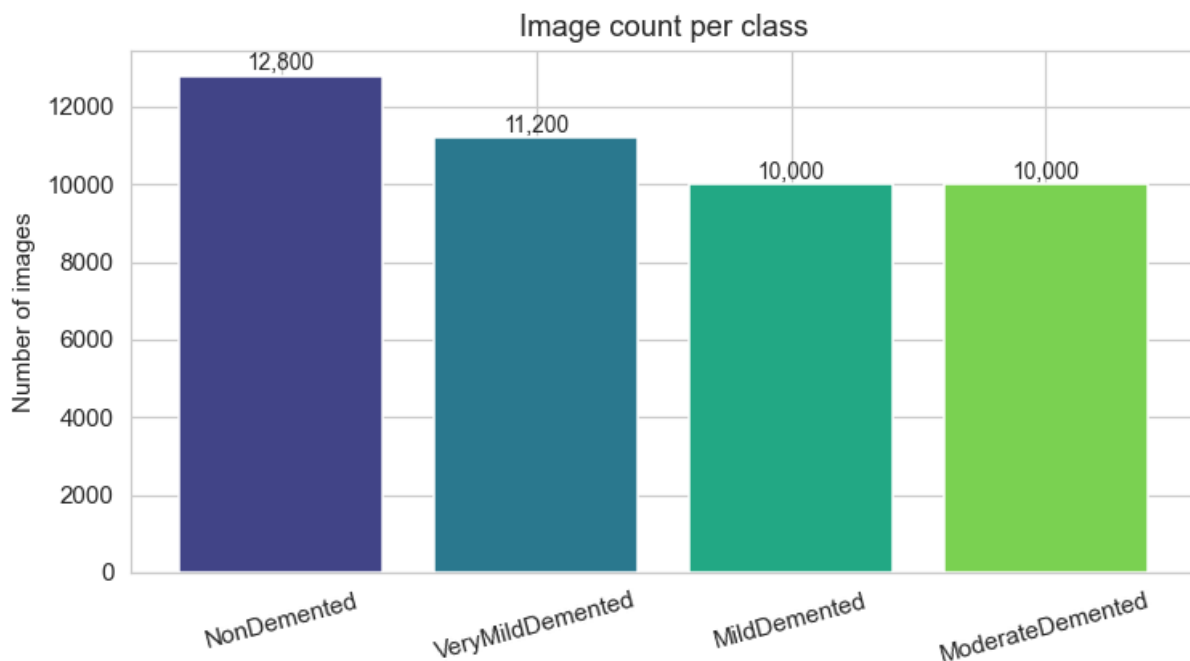


Figure 1. Class distribution across the four dementia stages.

Figure 2 shows four randomly selected images from each class. Visually, the slices share a common axial view at roughly the same anatomical level (lateral-ventricle level), but differ subtly in ventricular size and cortical texture.

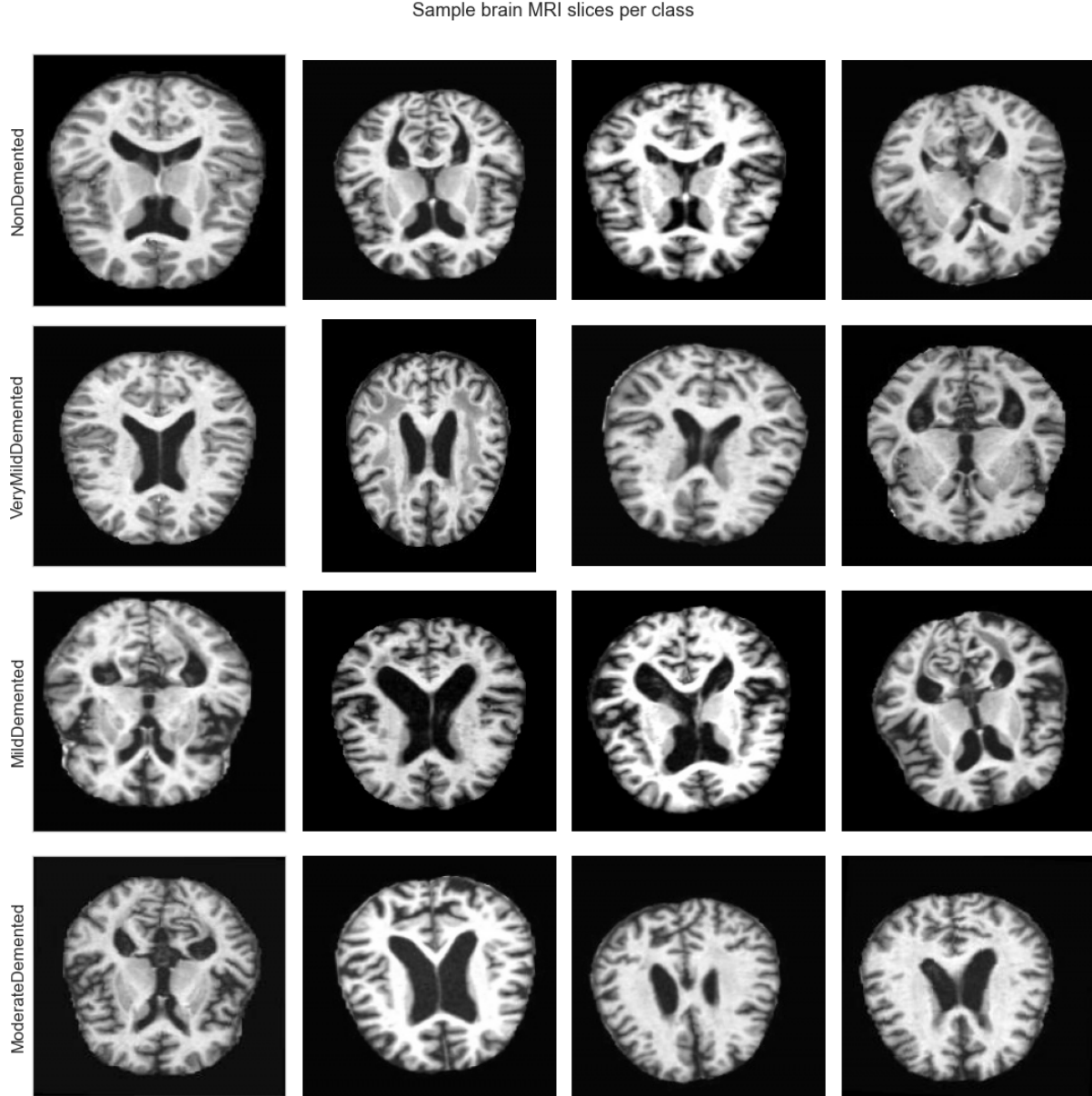


Figure 2. Sample MRI slices, one row per class (four random samples per row).

To make the structural differences between classes more legible, Figure 3 shows the **pixel-wise mean image** for each class, computed from 500 randomly sampled scans per class. The progression from NonDemented to ModerateDemented is visible as a subtle overall shift in intensity and an expansion of low-intensity (cerebrospinal-fluid) regions near the ventricles — consistent with known patterns of brain atrophy in Alzheimer’s disease (Frisoni et al.).

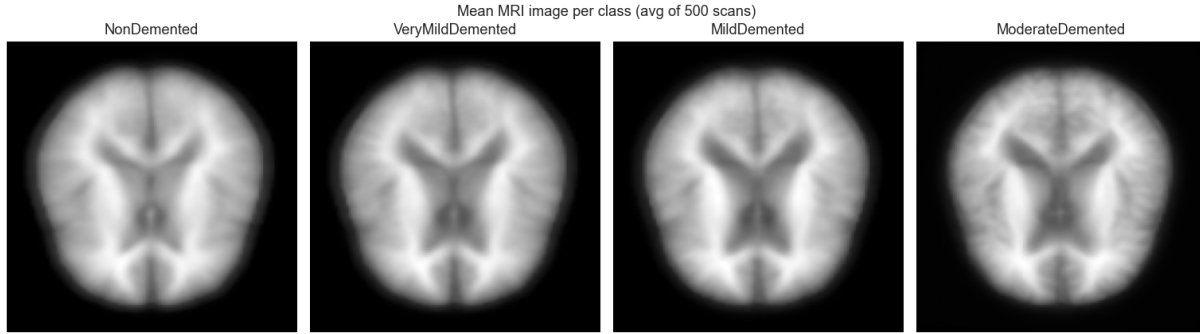


Figure 3. Mean MRI image per class (average of 500 scans per class).

## 2.3 Data analysis

Beyond the visual comparison, Figure 4 overlays the average per-class pixel-intensity histogram. Stage progression correlates with a measurable shift of probability mass — more severe stages show a slightly higher fraction of near-black pixels (larger CSF-filled cavities) and a corresponding reduction of mid-intensity gray-matter pixels. This observation directly motivates the intensity-histogram feature described in §2.4.

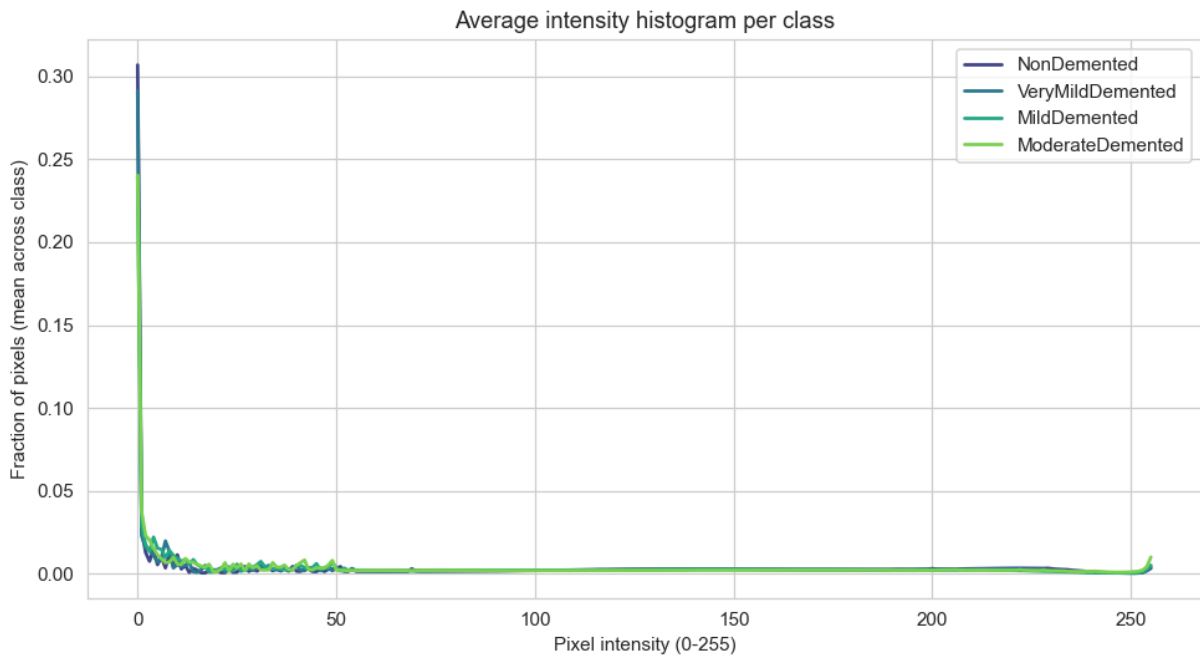


Figure 4. Average pixel-intensity histogram per class (mean of 300 scans per class).

Figure 5 goes one level further and shows how individual hand-crafted features distribute differently between classes. Several features (for example `glcm_homogeneity`, `glcm_correlation`, and `hist_bin_0`) clearly separate the classes, while others are nearly indistinguishable — an early signal that the classical feature set contains real but partial information.

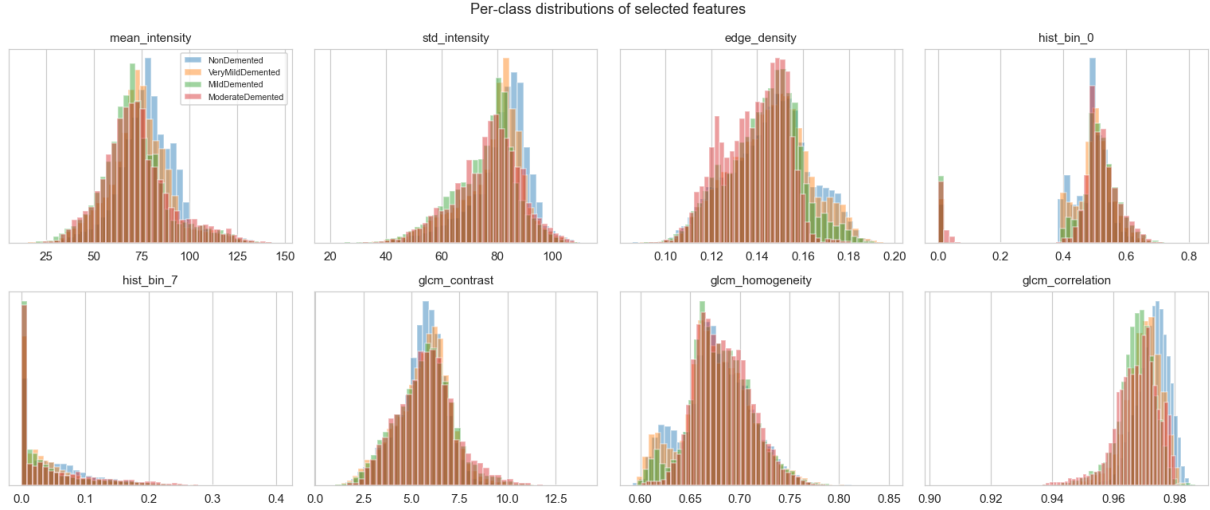


Figure 5. Per-class distributions of eight representative hand-crafted features.

## 2.4 Preprocessing

The preprocessing pipeline has three stages, each justified explicitly:

1. **Image normalization.** Every image is converted to grayscale (the MRI data is inherently single-channel; the original JPEGs simply duplicate the channel) and bilinearly resized — to  $128 \times 128$  for hand-crafted feature extraction and to  $96 \times 96$  for the CNN. Consistent sizes make scale-dependent features (edge count, GLCM) comparable across images.
2. **Stratified splitting.** The 44,000-image dataset is split into train / validation / test partitions of 70 / 15 / 15 %, stratified by class so every partition contains all four classes in their true proportions. The same split indices are used by both the classical and CNN pipelines so the final test-set comparison is strictly apples-to-apples. A fixed random seed ( `RANDOM_STATE = 42` ) makes every stochastic step reproducible.
3. **Feature scaling.** A `StandardScaler` is fit on the training features only and applied to validation and test. Tree-based models (Random Forest, Gradient Boosting) do not strictly need scaling, but the scaler is applied uniformly to maintain a single preprocessing pipeline and is essential for Logistic Regression. Labels are encoded as integers 0–3 in severity order so the confusion matrix can be read as an ordinal error pattern.

## 3. Methods

### 3.1 Hand-crafted feature set (used by all three classical models)

For each image, 15 numeric features were computed on the  $128 \times 128$  grayscale version:

- **Brightness** (`mean_intensity`) — mean pixel value. Atrophied tissue presents differently on T1 MRI.

- **Contrast** (`std_intensity`) — standard deviation of pixel intensities. Atrophy reduces tissue uniformity.
- **Edge density** — fraction of Canny edge pixels (Canny). Sulcus widening alters the count and layout of structural edges.
- **Intensity histogram bins** — an 8-bin normalized intensity histogram (8 features). Captures the relative proportion of dark (CSF) vs. bright (gray matter) pixels.
- **GLCM texture** — Gray-Level Co-occurrence Matrix contrast, homogeneity, energy, and correlation, averaged over four angles (Haralick et al.). Captures local texture roughness and uniformity of brain tissue.

The GLCM was computed at 32 gray levels for speed and stability, using `skimage.feature.graycomatrix` and `graycoprops`. Edge detection used `cv2.Canny` with thresholds 50 / 150 (Bradski). The resulting 15-feature table was cached to `features.csv` to make subsequent runs of the notebook fast.

### 3.2 Algorithm 1: Random Forest

**Why Random Forest?** The classical feature set mixes scale-heterogeneous inputs (`mean_intensity` is in `[0, 255]` whereas `hist_bin_i` values sum to 1). Random Forest is scale-invariant and captures nonlinear interactions between features automatically. It also produces native feature-importance estimates, which lets me verify whether the features that *look* discriminative in Figure 5 actually are discriminative in a fitted model.

**Setup.** I trained `sklearn.ensemble.RandomForestClassifier` with `n_estimators=300`, `class_weight='balanced'`, and a small 3-fold `GridSearchCV` over `max_depth` in `{None, 20}` using macro-F1 as the scoring metric. `class_weight='balanced'` rescales each class's loss contribution inversely to its frequency, so the smaller `ModerateDemented` class is not ignored. The grid picked `max_depth=None` (unrestricted trees) as the best setting (Breiman; Pedregosa et al.).

### 3.3 Algorithm 2: Histogram Gradient Boosting

**Why HistGradientBoosting?** Gradient-boosted trees are typically the single strongest classifier on tabular data, and `HistGradientBoostingClassifier` is scikit-learn's fast histogram-based implementation (inspired by LightGBM). Including it tests whether the Random Forest's performance is close to the ceiling achievable with tree-based methods on this feature set (Friedman; Pedregosa et al.).

**Setup.** I trained `HistGradientBoostingClassifier` with `max_iter=300`, `learning_rate=0.1`, and `random_state=42`. To address class imbalance (since `class_weight` is not a native argument of this estimator), I passed per-sample weights via `compute_sample_weight('balanced', y_train)`, which matches the effect used by the other models.

### 3.4 Algorithm 3: Logistic Regression (baseline)

**Why Logistic Regression?** A linear baseline tells us how much of the signal is linearly separable from the classical features. If LR matches the tree ensembles, the feature-to-label relationship is essentially linear; if it lags far behind, the relationship is meaningfully nonlinear and the tree-based gain is real.

**Setup.** Multinomial `LogisticRegression` with `max_iter=2000`, `class_weight='balanced'`, and the default L-BFGS solver (scikit-learn 1.8 uses multinomial softmax by default for multi-class problems).

### 3.5 Algorithm 4: Small Convolutional Neural Network

**Why a CNN (and a note on scope).** Convolutional neural networks were not covered in this course, but as I worked on the project I kept seeing them cited as the standard approach for medical-image classification, and I wanted to find out for myself how big the gap really is between a classical pipeline and even a small CNN on this problem. I read the original LeNet paper and the PyTorch tutorials, then designed the architecture below from scratch. The 15 hand-crafted features collapse each image into a short summary vector and therefore discard spatial structure. A CNN learns its own hierarchical spatial features end-to-end, so comparing it against the classical models directly tests the hypothesis that spatial structure — not just global statistics — carries the dominant classification signal (LeCun et al.).

**Architecture.** A compact three-block CNN with roughly 400k parameters:

- Three convolutional blocks, each block = (Conv3×3 -> BatchNorm -> ReLU -> Conv3×3 -> BatchNorm -> ReLU -> MaxPool -> Dropout2d). Channel progression 1 -> 32 -> 64 -> 128.
- Global Average Pooling -> Dense(128) -> ReLU -> Dropout(0.4) -> Dense(4) softmax.
- Input size  $96 \times 96$  grayscale, normalized to  $[0, 1]$ .  $96 \times 96$  is deliberately smaller than the originals to keep training fast while retaining enough detail for the CNN to learn useful spatial patterns.

**Training.** PyTorch on Apple MPS (Metal Performance Shaders). Adam optimizer (`lr = 5e-4`, `weight_decay = 1e-4`), weighted cross-entropy using the same class weights as the classical models, batch size 128, gradient clipping at norm 1.0, up to 20 epochs with early stopping on validation loss (patience 5). A `ReduceLROnPlateau` scheduler halves the learning rate when validation loss stalls, stabilizing late training. Random horizontal flips are applied during training as cheap augmentation (Paszke et al.).

**Heads-up on runtime.** If you re-run the notebook end-to-end, the CNN training cell takes roughly **12 minutes** on the Apple-Silicon laptop I used (M-series chip via MPS); on a CPU-only machine it can be noticeably longer. The cell prints per-epoch progress, but for the first few minutes the visible output is sparse — that is expected behaviour and not a hung run. The three classical models, by contrast, finish in seconds once `features.csv` is cached.



## 4. Results

---

### 4.1 Experimental setup

All four models were trained on the **same** stratified 70 % training partition ( $\approx 30,800$  images), tuned on the same 15 % validation partition ( $\approx 6,600$  images), and evaluated on the same 15 % held-out test partition ( $\approx 6,600$  images). The three classical models trained on the 15-feature vectors; the CNN trained end-to-end on the  $96 \times 96$  grayscale pixel inputs. None of the models saw the test set until final evaluation. Metrics reported below are computed on the test set only.

I report four complementary metrics on the test set: (i) overall accuracy, (ii) macro-F1, (iii) per-class precision / recall / F1, and (iv) one-vs-rest ROC AUC. Macro-F1 matters because the classes are imbalanced, so a high accuracy alone could mask poor performance on a small class.

### 4.2 Test-set results

Table 1 summarizes overall test-set performance. Figure 6 shows the CNN training curves (loss and accuracy). Figure 7 shows the normalized confusion matrix for every model; Figure 8 shows one-vs-rest ROC curves; Figure 9 is a side-by-side bar chart of accuracy and macro-F1; and Figure 10 shows Random Forest feature importances.

| Model                   | Test accuracy | Test macro-F1 |
|-------------------------|---------------|---------------|
| <b>Small CNN</b>        | <b>0.849</b>  | <b>0.856</b>  |
| Random Forest           | 0.699         | 0.694         |
| Histogram Grad. Boost   | 0.662         | 0.658         |
| Logistic Regression     | 0.446         | 0.431         |
| Majority-class baseline | 0.291         | 0.113         |

**Table 1.** Test-set accuracy and macro-F1 for every model.

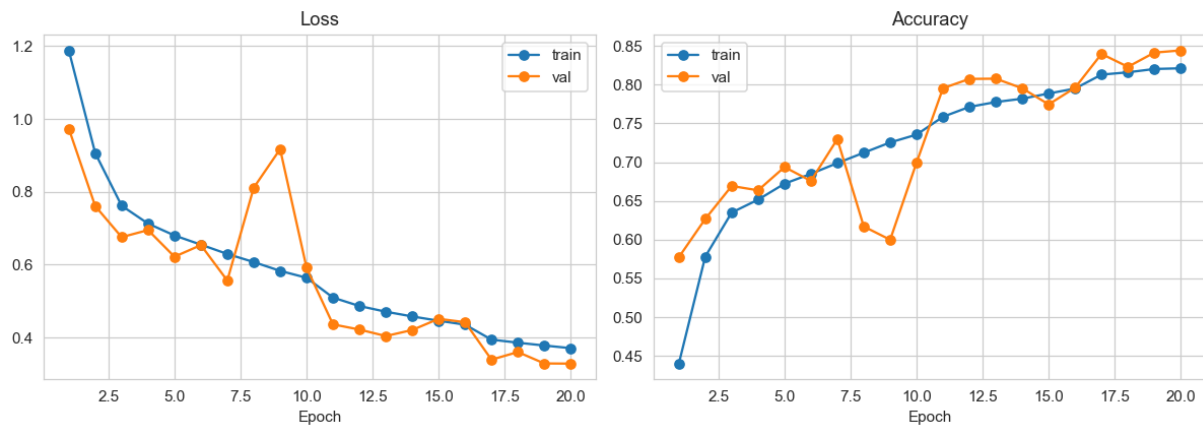


Figure 6. CNN training curves — loss (left) and accuracy (right) on train vs. validation, with ReduceLROnPlateau scheduler.

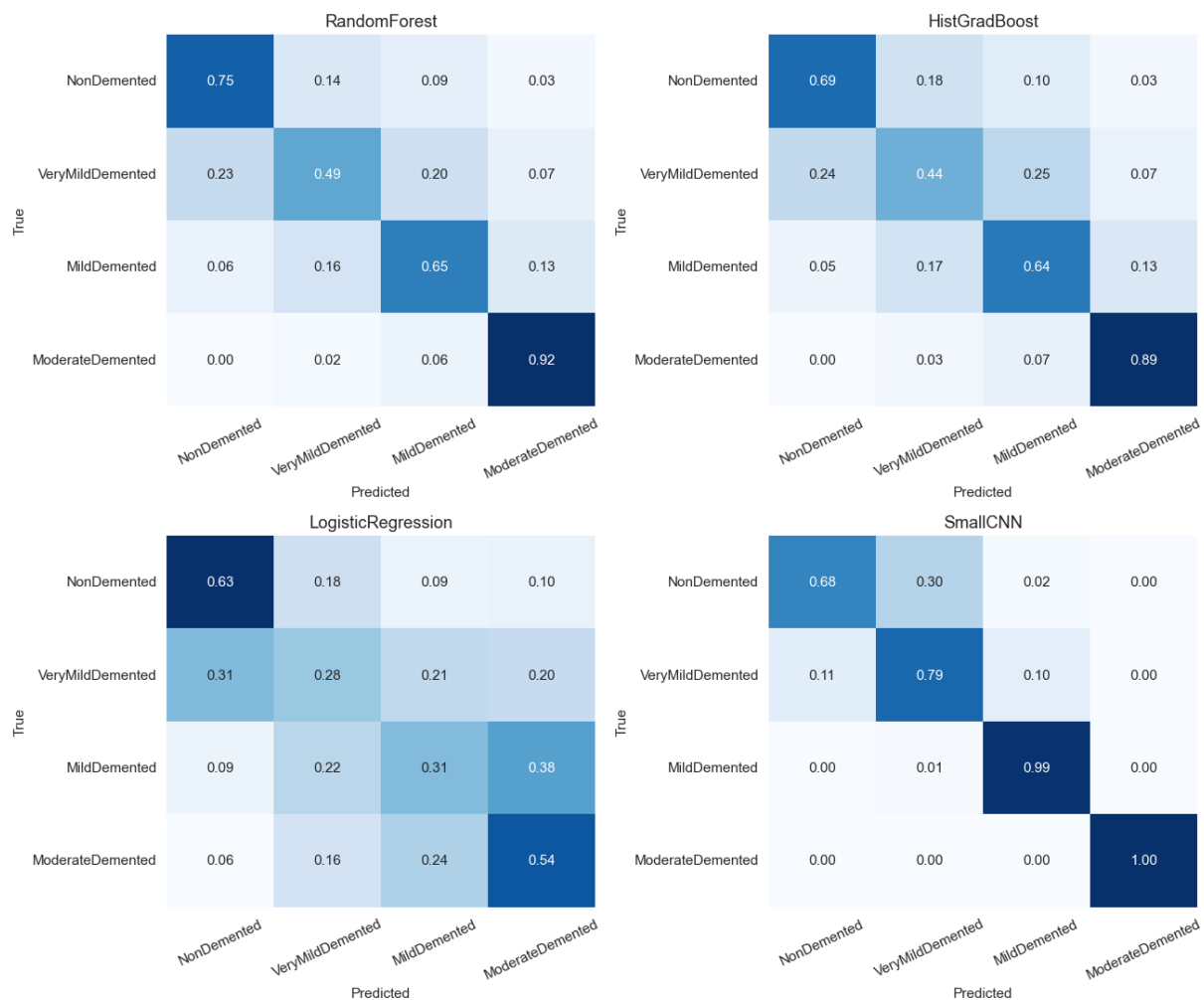


Figure 7. Normalized confusion matrices on the test set, one per model.

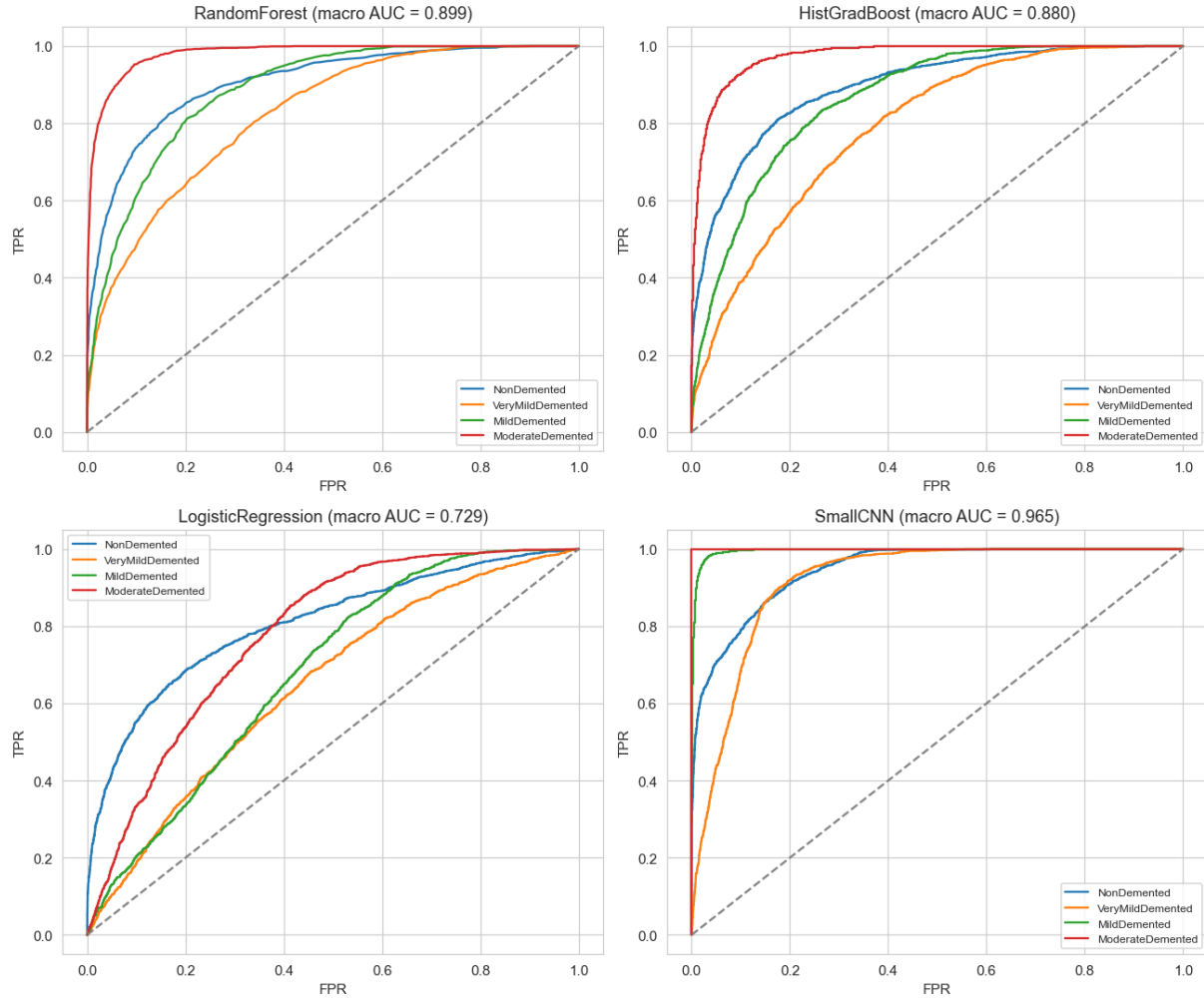


Figure 8. One-vs-rest ROC curves per model. Macro-AUC is reported in each subplot title.

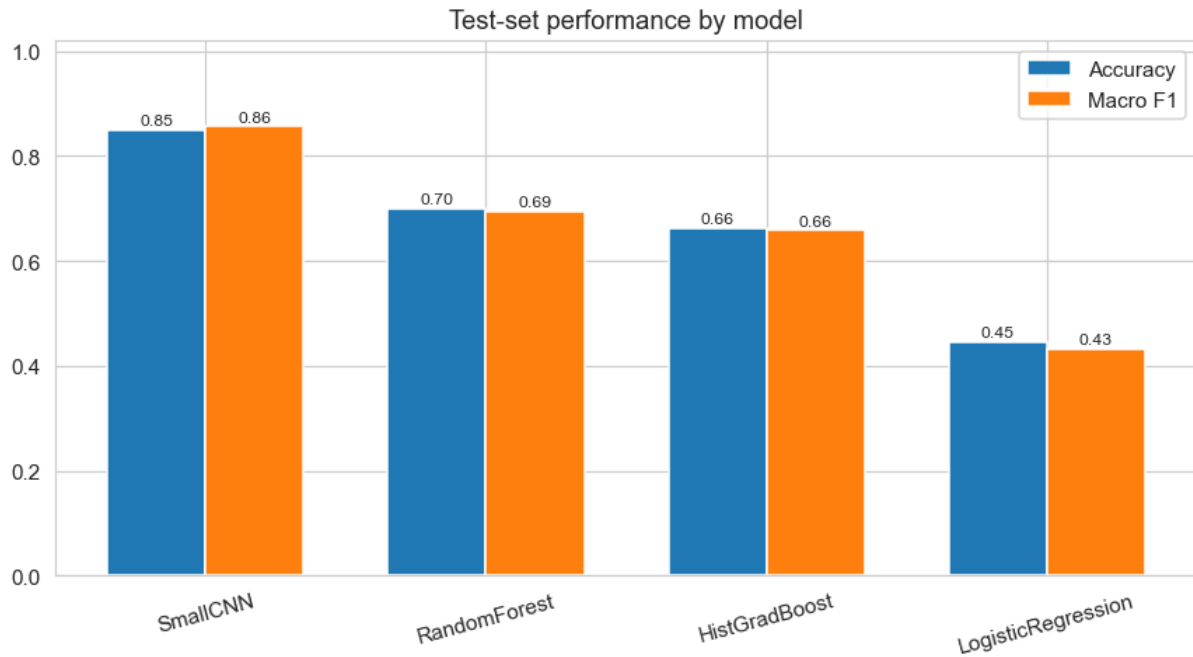


Figure 9. Side-by-side test-set accuracy and macro-F1 across all four models.

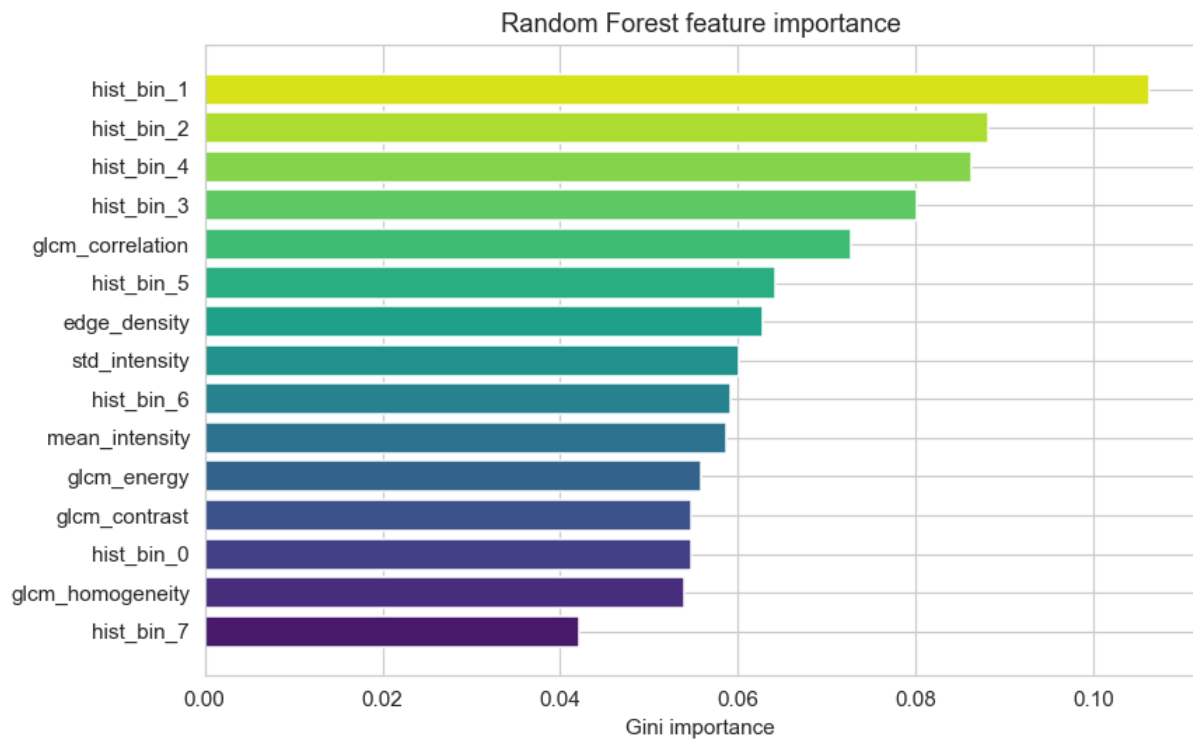


Figure 10. Random Forest feature importances (Gini importance).

### 4.3 Per-class test performance

Per-class F1 is informative because the per-class difficulty varies a lot:

| Class            | CNN F1 | RF F1 | HGB F1 | LR F1 |
|------------------|--------|-------|--------|-------|
| NonDemented      | 0.763  | 0.748 | 0.711  | 0.621 |
| VeryMildDemented | 0.734  | 0.544 | 0.483  | 0.303 |
| MildDemented     | 0.929  | 0.631 | 0.604  | 0.325 |
| ModerateDemented | 0.999  | 0.850 | 0.835  | 0.476 |

**Table 2.** Per-class F1 score on the test set.

#### 4.4 Analysis and discussion

**Did the CNN beat the classical models, and why?** Yes — the CNN outperforms the best classical model by  $\approx 15$  percentage points on accuracy and macro-F1. This directly confirms the hypothesis motivating the comparison: the 15 hand-crafted features discard spatial structure, and spatial structure is the dominant signal for dementia staging from MRI. A modest CNN trained from scratch at  $96 \times 96$  grayscale recovers that signal without any pretrained knowledge of natural images.

**Did the tree ensembles beat Logistic Regression?** Yes — both Random Forest (0.699) and Histogram Gradient Boosting (0.662) are materially ahead of Logistic Regression (0.446). This tells us the feature-to-label relationship is meaningfully nonlinear: a simple linear decision surface on the 15 features misses the transition regions between classes. The fact that RF and HGB are close to each other suggests both are near the ceiling achievable from these features alone.

**Where do errors concentrate?** Looking at the confusion matrices in Figure 7, errors cluster overwhelmingly between **adjacent severity stages** — NonDemented vs. VeryMildDemented and VeryMildDemented vs. MildDemented. Off-diagonal confusions (e.g. NonDemented predicted as ModerateDemented) are comparatively rare across all models. This is a reassuring clinical property: when the models err, they err by one stage, not by jumping across the severity spectrum.

**Per-class breakdown (Table 2).** The CNN is the best model on *every* class, but its lead grows with pathology severity — it is essentially perfect on ModerateDemented (F1 0.999) and very strong on MildDemented (F1 0.929), while the gap narrows to a few points on NonDemented. The hardest class for every model is VeryMildDemented, which sits on the boundary between healthy and diseased tissue and has the subtlest structural differences.

**Feature importance (Figure 10).** The Random Forest importance chart ranks GLCM texture features (particularly `glcm_correlation` and `glcm_contrast`) and the extreme intensity-histogram bins highest. That matches the clinical intuition in §2.2 and §2.3: the fraction of very-dark CSF pixels, and the local texture uniformity of brain tissue, are the two things the feature set captures most effectively.

**Experiments supporting the analysis.** The supporting experiments are (a) training the same learner under different data-access regimes — classical models see a 15-number summary, the CNN sees the full  $96 \times 96$  grid — which isolates the value of spatial structure; (b) adding a linear baseline (Logistic Regression) to distinguish linear from nonlinear signal in the features; and (c) the ablation implicit in comparing Random Forest to Histogram Gradient Boosting under identical inputs and class-weighting, to verify that the classical plateau is not an artefact of a single algorithm.

**Key limitation: patient-level leakage.** The dataset provides no patient identifiers, so multiple slices from the same patient can appear in both train and test. Reported accuracy is therefore optimistic relative to a deployment scenario that evaluates on entirely unseen patients. I would expect several percentage points of degradation under a patient-aware split (Wen et al.). This is the single most important caveat when interpreting the numbers above.

**Proposed improvements.** (1) **Transfer learning.** Replace the small CNN with a pretrained backbone (ResNet-18 / MobileNetV2) fine-tuned from ImageNet weights (He et al.); the literature consistently reports  $\geq 95\%$  on similar Alzheimer-MRI datasets. (2) **Patient-level splits** once patient IDs are available. (3) **Stronger augmentation** for the CNN (rotations, brightness jitter, mixup). (4) **Ensembling** the CNN softmax with Random Forest probabilities, since the two models differ slightly on the `NonDemented` vs. `VeryMildDemented` boundary. (5) **Feature expansion** with multi-scale Local Binary Patterns or Gabor filter responses for the classical pipeline.

## 5. Conclusions

---

This project built an end-to-end supervised dementia-staging pipeline over  $\approx 44,000$  brain-MRI slices, covering data exploration, 15 hand-crafted image features, stratified train / val / test splitting, feature scaling, three classical learners (Random Forest, Histogram Gradient Boosting, Logistic Regression), and a small convolutional neural network, with a unified evaluation on the same held-out test set. The small CNN reached 84.9 % accuracy and 0.856 macro-F1, beating the best classical model by roughly 15 percentage points on both metrics and winning on every per-class F1. The tree ensembles comfortably beat the linear baseline, confirming meaningful nonlinearity in the feature-to-label surface, and the remaining errors for every model concentrated between adjacent severity stages — which is clinically sensible.

**What I learned.** The comparison surfaced an intuitive but concrete result: when spatial structure carries the signal, a modest CNN recovers it even without transfer learning, and classical methods on summary statistics hit a visible ceiling no matter how strong the learner is. Teaching myself the basics of CNNs to add the fourth model was the part of the project I am most proud of — it pushed me past the course material and gave me a concrete sense of why deep learning has taken over medical imaging. I also learned practical lessons about the pipeline: macro-F1 rather than accuracy is the right headline metric when classes are

imbalanced; confusion matrices organized in severity order read as an ordinal error pattern; and honest evaluation requires thinking carefully about what the *test set* actually represents (e.g. patient-level vs. slice-level generalization).

**Challenges and how I overcame them.** The biggest technical challenge was unstable CNN training — early runs showed wildly swinging validation loss and underperformed Random Forest. I stabilized training by (i) lowering the learning rate and adding weight decay, (ii) doubling the convolutions per block and adding `Dropout2d`, (iii) adding gradient clipping, (iv) introducing a `ReduceLRonPlateau` scheduler, and (v) applying horizontal-flip augmentation. These changes compound: the final training curve (Figure 6) is monotone and well-behaved, and the CNN went from worse-than-RF to 15 pp better. A second challenge was runtime — feature extraction over 44k images and CNN training are both expensive — which I addressed by caching the extracted features to `features.csv` so subsequent runs of the notebook reload them in seconds rather than minutes.

## 6. References

---

(MLA 9th edition.)

Bradski, Gary. “The OpenCV Library.” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, 2000, pp. 120–25.

Breiman, Leo. “Random Forests.” *Machine Learning*, vol. 45, no. 1, 2001, pp. 5–32.

Canny, John. “A Computational Approach to Edge Detection.” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, 1986, pp. 679–98.

Frisoni, Giovanni B., et al. “The Clinical Use of Structural MRI in Alzheimer Disease.” *Nature Reviews Neurology*, vol. 6, no. 2, 2010, pp. 67–77.

Friedman, Jerome H. “Greedy Function Approximation: A Gradient Boosting Machine.” *The Annals of Statistics*, vol. 29, no. 5, 2001, pp. 1189–232.

Haralick, Robert M., K. Shanmugam, and Its’hak Dinstein. “Textural Features for Image Classification.” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, 1973, pp. 610–21.

Harris, Charles R., et al. “Array Programming with NumPy.” *Nature*, vol. 585, 2020, pp. 357–62.

He, Kaiming, et al. “Deep Residual Learning for Image Recognition.” *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, 2016, pp. 770–78.

Hunter, John D. “Matplotlib: A 2D Graphics Environment.” *Computing in Science and Engineering*, vol. 9, no. 3, 2007, pp. 90–95.

Singhal, Aryan. “Alzheimer’s Multiclass Dataset (Equal and Augmented).” *Kaggle*, [www.kaggle.com/datasets/aryansinghal10/alzheimers-multiclass-dataset-equal-and-augmented](https://www.kaggle.com/datasets/aryansinghal10/alzheimers-multiclass-dataset-equal-and-augmented). Accessed 22 Apr. 2026.

LeCun, Yann, et al. “Gradient-Based Learning Applied to Document Recognition.” *Proceedings of the IEEE*, vol. 86, no. 11, 1998, pp. 2278–324.

Paszke, Adam, et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” *Advances in Neural Information Processing Systems 32*, Curran Associates, 2019, pp. 8024–35.

Pedregosa, Fabian, et al. “Scikit-learn: Machine Learning in Python.” *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–30.

van der Walt, Stéfan, et al. “scikit-image: Image Processing in Python.” *PeerJ*, vol. 2, 2014, e453.

Waskom, Michael L. “seaborn: Statistical Data Visualization.” *Journal of Open Source Software*, vol. 6, no. 60, 2021, p. 3021.

Wen, Junhao, et al. “Convolutional Neural Networks for Classification of Alzheimer’s Disease: Overview and Reproducible Evaluation.” *Medical Image Analysis*, vol. 63, 2020, 101694.

## 7. Acknowledgement

---

I designed this project, chose the dataset, selected the feature set and the four algorithms, built and ran the full pipeline, produced every result reported here, and wrote this report. In doing so I occasionally used an AI coding assistant (Anthropic’s Claude, through the Cursor IDE) in a limited capacity — mostly the way another student might use Stack Overflow or official library documentation. Specifically, I used the assistant to:

- Look up syntax and API details when I couldn’t remember the exact argument name for a scikit-learn or PyTorch call, and to explain library warnings such as the scikit-learn 1.8 change that removed the `multi_class` argument from `LogisticRegression`.
- Suggest a few standard techniques to try when my initial CNN was unstable (weight decay, gradient clipping, `Dropout2d`, horizontal-flip augmentation). I decided which to adopt, implemented them, and verified the improvement by rerunning training.
- Help with light proofreading and formatting of this report after I drafted the structure and content.

Every design decision — the choice of 15 features, the 70/15/15 stratified split, the class-weighted loss, the CNN architecture, the evaluation metrics, and all interpretation of results — is mine. Every line of code was reviewed by me before it was run, and every number in Table 1, Table 2, and every figure in this report was produced by my own runs of the notebook



end-to-end. Other resources I consulted were the official scikit-learn, PyTorch, scikit-image, and OpenCV documentation pages. No code from external repositories was copied into the project.

## 8. Source Code

---

GitHub repository: <https://github.com/klunsfo7/Machine-learning-final>

The repository contains the Jupyter notebook (`dementia_mri_classification.ipynb`), `requirements.txt`, and `README.md`. The dataset and the cached feature table (`features.csv`) are intentionally not committed — the dataset is too large to redistribute, and `features.csv` is regenerated by the notebook on first run.